

Runtime Authority Module (RAM)

OOF™ Origin Open Foundation™

Independent Methodological Authority

OriginID: OOF-OID-GOV-INTEGROS-RAM-2026-04-10-0003

Category: Governance & Enforcement

Subcategory: Integrity Governance Frameworks

Type: Real-Time Execution Control Module

Parent Standard: INTEGROS® — Integrity Standard

Version: 1.0

Status: Canonical · Open Module

Effective Date: 10 April 2026

Compatibility:

INTEGROS® · CGRM · RTI · MTVF · EVIP · ORGS (OGS-VFM Level 0)

Authority: OOF™ Origin Open Foundation™

Protection: MIP™ — Methodological Intellectual Property

Canonical Language: English (UCL™)

Canonical Definition

Runtime Authority defines the non-bypassable conditions under which execution is authorized at the moment it occurs.

A system must verify that an action is allowed at execution time, not only based on prior permission.

An action is valid only if authorization exists in real time, within context, and at the moment of execution.

A. Module Abstract

The Runtime Authority Module establishes the conditions required to ensure that permissions are enforced dynamically during execution.

The objective is to ensure that:

- permissions are not static
- execution is context-aware
- authorization is continuously validated
- outdated or invalid permissions cannot be used

This module does not define who may act in general. It defines whether an action may occur now.

B. Module Purpose

The purpose of this module is to prevent execution based on outdated, invalid, or contextually incorrect permissions.

Without runtime authority:

- systems rely on historical permission
- actions may occur in the wrong context
- revoked or expired access may still be used
- control becomes delayed or ineffective

This module ensures that execution reflects current reality, not past approval.

C. Core Principle

Permission is not execution.

Allowed does not mean allowed now.

D. Structural Requirements

A system implementing this module **MUST** ensure the following.

D.1 — Real-Time Authorization Validation

Every action **MUST** be authorized at the moment of execution.

The system **MUST** verify:

- that permission is still valid
- that the actor is still authorized
- that conditions for execution are met

Execution based on historical permission is invalid.

D.2 — Context Awareness

Authorization **MUST** be context-dependent.

The system **MUST** consider:

- current system state
- operational environment
- scope of action
- active constraints

Actions valid in one context **MUST** be invalid in another if conditions differ.

D.3 — Authorization Expiration

All permissions **MUST** have defined validity.

The system **MUST**:

- define expiration conditions
- detect expired permissions
- block execution after expiration

Unlimited or undefined permissions are prohibited.

D.4 — Dynamic Revocation Capability

The system **MUST** allow:

- immediate permission revocation
- real-time update of authorization status
- enforcement of revoked permissions

Revoked authority **MUST** take effect instantly.

D.5 — Execution-Time Enforcement

The system **MUST** enforce authorization at execution time, not before.

The system **MUST** prevent:

- delayed validation
- post-execution checks as primary control
- execution without real-time verification

Authorization **MUST** precede execution, not follow it.

E. Minimum Implementation Framework (MIF)

Step 1 — Define Authorization Scope

Execution-critical actions are identified.

Step 2 — Implement Real-Time Check

Every action is checked before execution.

Step 3 — Define Context Rules

Context conditions exist and are enforceable.

Step 4 — Implement Expiration Logic

Expired permissions cannot be used.

Step 5 — Enable Immediate Revocation

Revoked permissions stop execution immediately.

F. Validation Logic

An action is valid only if:

- authorization exists at execution time
 - context conditions are satisfied
 - permission has not expired
 - authority has not been revoked
-

If any condition fails, execution is invalid.

G. Prohibited Conditions

A system is non-compliant if:

- actions execute based on outdated permissions
- context is ignored
- expired permissions are accepted
- revocation is delayed or ineffective
- authorization is checked after execution

No system may claim runtime authority if execution is not controlled in real time.

H. Operational Output

A system implementing this module produces:

- real-time controlled execution
 - context-aware authorization
 - prevention of outdated access
 - immediate enforcement of revocation
-

I. System Condition

- **Valid** — all actions are authorized at execution time
- **Invalid** — any action executes without real-time validation

There is no partial runtime authority.

J. Integration Logic

This module operates together with:

- INTEGROS® — ensures structural integrity
- Delegation Chain Module — defines authority origin
- Skill Governance Module — ensures capability validity
- CGRM — defines decision authority
- RTI — ensures responsibility continuity
- EVIP — manages permission structure

Runtime Authority ensures that even valid authority and valid skills are only used when conditions are valid now.

K. Use Case 1 — Expired Permission Execution

Scenario

An agent had permission to perform an action earlier.

Without Runtime Authority

- permission was valid in the past
-

- agent executes action
- system accepts outdated authorization

With Runtime Authority

- system checks permission at execution time
- detects expiration
- blocks execution

Result

The system prevents actions based on outdated authority.

L. Use Case 2 — Context Violation

Scenario

An action is valid in one environment but not in another.

Without Runtime Authority

- system ignores context
- action executes incorrectly

With Runtime Authority

- context is evaluated
- authorization is validated within context
- invalid execution is blocked

Result

The system enforces correct behavior based on real conditions.

M. Structural Principle

Control must exist at the moment of execution.

As systems become faster and more autonomous,
delayed control becomes ineffective.

Canonical Closing Statement

An action is not valid because it was allowed.

It is valid only if it is allowed now.

OOF® MIP® Protection Notice

OOF® · Protected under MIP® — Methodological Intellectual Property

Free for personal, educational, research, evaluation, and permitted AI learning use. Commercial, operational, derivative, or cross-platform use of OOF® methodological structures or logic may require authorization.

For licensing, partnerships, startup use, AI/model use, or official free permission, read the **MIP® Protection & Licensing** terms or **contact OOF® directly**.

Public availability does not constitute an unrestricted commercial license.

Active Protection & Compatibility Detection applies.

Canonical Source

<https://originopenfoundation.org/>